

RAPPORT DE PROJET – TBA

Text-Based Adventure

Page de garde

Type de projet :

Projet académique – Développement logiciel

Description :

Jeu d'aventure textuel développé en Python

Réalisé par :

- **Amine El Mouttaki**
- **Youssouf Timit**

Langage de programmation :

Python

Année universitaire :

2025 – 2026



Rapport détaillé sur le projet TBA

Introduction

Le projet **TBA** (*Text-Based Adventure*) est un jeu d'aventure textuel développé en Python. Il s'agit d'un jeu dans lequel le joueur explore un labyrinthe, collecte des objets, interagit avec des personnages non-joueurs (PNJ) et accomplit des quêtes afin de s'échapper.

L'objectif principal du jeu est de vaincre deux ennemis clés, dans un ordre précis : le **Fantôme**, puis le **Gardien**. Une fois ces deux adversaires vaincus, le joueur remporte la partie.

Le projet est conçu de manière modulaire, avec plusieurs classes interconnectées, permettant une séparation claire des responsabilités. L'interface est exclusivement textuelle, et toutes les interactions se font via des commandes saisies par l'utilisateur dans la console.

Architecture et structure du projet

Le code est structuré autour de plusieurs modules Python, chacun correspondant à une classe centrale du jeu. Cette organisation favorise la lisibilité, la maintenance et l'extensibilité du projet.

- **game.py**
Classe **Game** qui gère l'environnement global du jeu, la configuration initiale, la boucle principale et l'exécution des commandes.
- **room.py**
Classe **Room** représentant les lieux (pièces) du labyrinthe, avec leurs descriptions, sorties, objets et PNJ.
- **player.py**
Classe **Player** gérant le joueur, son inventaire, sa santé, son historique de déplacements et ses actions.
- **command.py**
Classe **Command** définissant les commandes disponibles, leurs descriptions d'aide et leurs actions associées.
- **actions.py**
Classe **Actions** contenant toutes les méthodes statiques permettant d'exécuter les actions du jeu (déplacement, prise d'objets, dialogues, combats, etc.).
- **item.py**
Classe **Item** représentant les objets du jeu, avec un nom et une description.
- **character.py**
Classe **Character** (utilisée comme **NPC**) gérant les personnages non-joueurs, leurs dialogues, leur santé et leurs déplacements.

- **quest.py**
Classe `Quest` implémentant le système de quêtes avec objectifs, vérification de complétion et récompenses.

Le point d'entrée principal du programme est la fonction `main()` dans le fichier `game.py`, qui instancie un objet `Game` et lance la méthode `play()`.

Classes et leurs rôles détaillés

Classe `Game`

- **Responsabilités :**
 - Initialisation du jeu (pièces, objets, PNJ, quêtes)
 - Gestion des commandes
 - Boucle principale du jeu
 - Déplacement aléatoire des PNJ
- **Méthodes clés :**
 - `setup()` : configuration complète du jeu
 - `move_npcs()` : déplacement aléatoire des PNJ (20 % de chance par tour)
 - `process_command()` : analyse et exécution des commandes
 - `play()` : boucle principale
- **Attributs :**
`rooms, commands, player, quests, finished, ghost_defeated`

Classe `Room`

- **Responsabilités :**
 - Représentation des lieux du jeu

- Gestion des objets et PNJ présents
 - **Méthodes clés :**
 - `get_long_description()`
 - `add_item(), remove_item(), get_item_by_name()`
 - `add_npc(), remove_npc(), get_npc_by_name()`
 - **Attributs :**
`name, description, exits, items, npcs`
-

Classe **Player**

- **Responsabilités :**
 - Gestion de l'état du joueur et de ses actions
 - **Méthodes clés :**
 - `move()`
 - `get_history()`
 - `add_item_to_inventory()`
 - `remove_item_from_inventory()`
 - `get_item_from_inventory()`
 - `take_damage()`
 - **Attributs :**
`name, current_room, history, inventory, health, direction_aliases`
-

Classe **Command**

- **Responsabilités :**
 - Définition des commandes et liaison avec les actions
 - **Méthodes clés :**
 - `execute()`
 - **Attributs :**
`command_word, help_string, action, number_of_parameters`
-

Classe **Actions**

- **Responsabilités :**
 - Implémentation de toutes les actions possibles dans le jeu
 - **Méthodes clés :**
 - `go()`, `take()`, `drop()`
 - `talk()`
 - `attack()`
 - `use()`
 - `quests()`
 - Autres : `help`, `quit`, `look`, `inventory`, `status`, `listPNJ`, `historik`, `back`, `wait`
 - **Logique spéciale :**
 - Le joueur doit vaincre le Fantôme avant de pouvoir vaincre le Gardien.
-

Classe **Item**

- **Responsabilités :**
 - Représentation simple des objets
 - **Attributs :**
`name, description`
-

Classe **Character** (NPC)

- **Responsabilités :**
 - Gestion des PNJ, dialogues et combats
 - **Méthodes clés :**
 - `talk()`
 - `take_damage()`
 - `move_to_room()`
 - **Attributs :**
`name, description, dialogues, health, is_alive, talked`
-

Classe **Quest**

- **Responsabilités :**
 - Gestion du système de quêtes
 - **Méthodes clés :**
 - `check_completion()`
 - **Attributs :**
`name, description, quest_type, target, completed, reward`
-

Mécaniques de jeu

Exploration et navigation

- 8 pièces interconnectées
- Déplacements via directions cardinales
- Historique des déplacements avec retour possible

Objets et inventaire

- 6 objets : clé, lampe, livre, torche, corde, pioche
- Commandes `take`, `drop`, `use`

PNJ et interactions

- Marchand, Fantôme (30 PV), Gardien (100 PV)
- Dialogues cycliques
- Combats avec contre-attaques

Système de quêtes

- Explorer le Bureau
- Collecter la Clé
- Parler au Marchand

Conditions de victoire

- Fantôme vaincu → Gardien accessible
- Santé initiale : 100 PV

Analyse de la qualité du code

Points forts

- Modularité
- Lisibilité
- Extensibilité
- Logique de jeu cohérente

Limites

- Interface textuelle
- IA simple
- Peu de gestion d'exceptions
- Absence de tests unitaires

Améliorations possibles

- Tests avec `pytest`
- Interface graphique
- Sauvegarde de partie
- PNJ plus intelligents
- Événements dynamiques

Conclusion

Le projet **TBA** est une implémentation solide d'un jeu d'aventure textuel en Python, démontrant une bonne maîtrise de la programmation orientée objet. Il propose une expérience complète incluant exploration, quêtes, dialogues et combats, tout en restant extensible. Malgré une interface textuelle simple, il constitue une excellente base pour des développements futurs plus avancés.